

ГЛАВА III. ЭТА (Элементы Теории Алгоритмов)

Имея в виду задачу под названием ИИ, нельзя обойти такое понятие как

алгоритм.

Обычно алгоритмом называют набор команд или инструкций, позволяющий механическим («машинным») образом достигать какие-либо цели.

Наши первые шаги в этом разделе будут связаны с записью алгоритмов в форме, понятной всем нам, человечеству, компьютерам. Тем самым, нужен **алфавит алгоритмов**. Сразу возникают вопросы: а можно ли все, что мы называем алгоритмом (в самой широкой форме), записать в соответствии с высказанным желанием? Каким (насколько большим) может или должен быть такой алфавит?

Пример. Пчела-разведчица передает всему улью инструкцию (куда лететь за медом) БЕЗ АЛФАВИТА в нашем понимании.

Ясно, что гипотетически можно представить совершенно непредставимые (пока) алгоритмы и способы их фиксации (записи), на которые, возможно, не хватит уже используемых алфавитов.

И здесь уже можно сказать, что за неполный век существования и развития Дискретной Математики (опирающейся на современные идеи Математической Логики) выработалось следующее отношение к термину «алгоритм»:

1) понятие это - неопределяемое, первичное (такое, как, например, точка или прямая в «устоявшейся» математике);

1) это НЕ мешает разрабатывать различные «частичные» формализации этого понятия и описывать в их рамках то, что поддается таким описаниям.

Жизнь показывает, что поддается многое. А кое-что НЕ поддается, и это можно аккуратно доказать. Например, в качестве комментария к прошлой лекции, в которой было сказано, что никакая (достаточно содержательная) аксиоматическая теория, например, Арифметика, не может быть одновременно полной и непротиворечивой. Одно из (доказательных) объяснений этой ситуации, существующих в математике, состоит в том, что:

а) понятие доказуемой (выводимой) формулы является **алгоритмизируемым**;

б) понятие верной формулы (в содержательной теории) таковым НЕ является.

А потому мы начинаем изучение одной из формализаций понятия «алгоритм», называемой «Машиной Тьюринга».

Замечание. Известны и другие частичные формализации понятия алгоритма, например, такие как Машина Поста и Нормальные Алгорифмы Маркова. Имеется в теории алгоритмов еще одно важное (практическое !) соображение, называемое тезисом Черча (Church). Оно утверждает, что различные частичные формализации фактически описывают одно и то же, а именно, *частично-рекурсивные функции*. Что это такое - мы узнаем на третьей лекции по этой теме. А пока приступим к формальному описанию Машины Тьюринга.

§1. Понятие Машины Тьюринга (МТ)

Итак, МТ — это прибор, состоящий из ленты, управляющего устройства и считывающей головки (см. Рис. 1).

Лента тянется бесконечно в обе стороны.

$\mathcal{A}$ - внешний алфавит. У нас чаще всего будет $\mathcal{A} = E = \{0, 1\}$.

На ленте влево и вправо в бесконечность уходят нули, заполняющие отдельные ячейки. Лишь в **рабочей зоне** может быть некая запись в ячейках; каждая запись — это слово в алфавите E , записанное по одному символу в ячейке.

$Q = \{q_0, q_1, q_2, \dots, q_{r-1}\}$ — внутренний алфавит (алфавит состояний); $|Q| = r \geq 2$, иначе - неинтересно.

Машина работает в дискретные моменты времени и в каждый из таких моментов находится в одном из своих состояний.

Принцип работы МТ

Задается начальное состояние машины q_1 . На ленте предполагается записанным некоторое (двоичное) слово; головка машины в начале работы расположена напротив первого **ненулевого** символа на ленте.

В каждый такт (в каждую «секунду») работы машина «обрабатывает» ячейку на ленте, на которую направлена ее рабочая головка. Обработка подразумевает чтение, «раздумья» и новую запись взамен старой в обраба-

тываемую ячейку. Каждый такт включает в себя смену состояния головки на одно из состояний множества Q (в т.ч., возможно сохранение состояния). Завершением такта работы является либо сдвиг головки машины на одну ячейку (вправо или влево), либо головка остается на месте.

На каждом (первом, втором и последующих) такте совершаются операции, описанные выше.

В итоге у вас уже должно появиться понимание, что машина перерабатывает записи на ленте (слова) в другие слова. Можно предложить следующее формальное описание происходящего с машиной. В каждый момент времени t мы имеем три сигнала от МТ:

$z(t) = F(x(t), q(t-1))$ — выходящий сигнал, т.е. новый символ на ленте,

$q(t) = G(x(t), q(t-1))$ — состояние головки,

$d(t) = H(x(t), q(t-1))$ — сдвинуться на одну ячейку или остаться на месте.

Здесь $x(t), z(t) \in \mathcal{A}$, $q(t) \in Q$, $d(t) \in \{L, R, S\}$, т.е. для описания работы МТ используются три алфавита.

Можно те же три строки описания работы МТ записать короче:

$$z : \mathcal{A} \times Q \rightarrow \mathcal{A}, \quad q : \mathcal{A} \times Q \rightarrow Q, \quad d : \mathcal{A} \times Q \rightarrow \{S, L, R\}.$$

Замечание. Первые две строки из этого описания совпадают с описанием ОДФ из первой главы нашего семестра.

Вопрос: Как долго должна работать машина ? Нужны договоренности о способах ее остановки, хотя и вариант бесконечно долгой ее работы тоже допускается.

Для остановки МТ договоримся считать, что при переходе машины в состояние q_0 происходит именно остановка МТ (все другие состояния головки — рабочие).

Как заставить машину выполнять отдельные действия ? Для этого используются *команды* и *программы*.

Определение 1. Команда для МТ — это упорядоченный набор из 5 символов: два первых фиксируют

(состояние головки до начала текущего такта работы; читаемый символ на ленте);

три последних — это

(новое состояние головки; символ, записываемый на ленте; символ смещения).

Пример 1. $\mathcal{A} = E = \{0, 1\}$. Команда:

$$q_1 \ 1 \ q_2 \ 1 \ R \quad \text{или} \quad q_1 1 q_2 1 R$$

Определение 2. Программа для МТ — это совокупность отдельных команд (упорядоченность команд необязательна).

Пример 2. $\mathcal{A} = E = \{0, 1\}$.

$$\begin{array}{l} q_1 1 q_2 1 R \\ \Pi : \quad q_1 0 q_1 1 S \\ \quad q_2 1 q_0 0 R \\ \quad q_2 0 q_1 1 L \end{array}$$

Что делает эта программа ? Разумеется, универсального ответа нет. Надо знать к чему мы ее применяем, к какой записи на ленте ?

Пусть на ленте записано слово

$$1011.$$

Тогда можно получить конкретный ответ на поставленный вопрос. Для его обсуждения привлекаются еще некоторые понятия и договоренности.

Определение 3. Конфигурация на ленте - это запись слова на ленте в конкретный момент времени с указанием положения головки МТ.

При описании конфигураций головку размещают **слева от обрабатываемой ячейки**.

Опишем работу программы из Примера 2 со словом 1011.

$$q_1 1011 \rightarrow 1 q_2 011 \rightarrow q_1 1111 \rightarrow 1 q_2 111 \rightarrow 10 q_0 11.$$

Эта цепочка конфигураций означает, что

$$\Pi : 1011 \rightarrow 1011.$$

Подчеркнем, что здесь работа со словом произведена, закончена и результатом является итоговая запись на ленте (после остановки машины), совпадающая с самим исходным словом.

Упражнение. Что делает та же программа с другими словами, например, с

$$111, 1001, 110011 ?$$

Рассмотренный пример и упражнение должны вас научить чувствовать начальные понятия теории алгоритмов. Но, конечно, можно ставить и более сложные задачи с заранее озвученными целями.

Пример 3. Построить МТ (написать программу), т. чтобы $11 \rightarrow 1111$ (удвоение слова $11 = 1^2$).

Решение (один из возможных вариантов).

Первая команда:

$$q_1 1 q_1 1 R$$

Комментарии в терминах конфигураций:

$$q_1 11 \rightarrow 1 q_1 1 \rightarrow 11 q_1 = 11 q_1 0000 \dots \rightarrow ?$$

Вторая команда:

$$q_1 0 q_1 1 R$$

Что произойдет на ленте ? ...

Вывод: такая вторая команда нам НЕ нужна. Исправляем:

Исправленная вторая команда :

$$q_1 0 q_2 1 R : \quad 11 q_1 000 \dots \rightarrow 111 q_2 00 \dots$$

Третья команда :

$$q_2 0 q_3 1 R : \quad 111 q_2 000 \dots \rightarrow 1111 q_3 00 \dots$$

Вывод: дело сделано, осталось культурно оформить сказанное:

$$\begin{array}{l} q_1 1 q_1 1 R \\ \text{П : } q_1 0 q_2 1 R \quad \text{или} \quad \text{П}_1 : q_1 0 q_2 1 R \\ q_2 0 q_0 1 R \quad \quad \quad q_2 0 q_0 1 S \end{array} \quad \text{— итоговые программы.}$$

Вопрос: какая разница между двумя этими программами ? В смысле получаемых записей на ленте — никакой. А вот в конфигурациях разница видна:

$$1111 q_0 \quad \text{или} \quad 111 q_0 1$$

Эта разница будет существенной при написании более сложных программ. А вам, возможно, уже ясно, что современные ИИ (компьютеры) можно собирать на принципах объединения большого количества примитивных МТ типа рассмотренной. А раз так, то нужно позаботиться о «стыковках» разных МТ. Поэтому принципы «культурной работы» с МТ подсказывают, что программа П предпочтительнее, чем П₁.

И вообще, будем считать программу **правильной**, если после ее применения (к интересующим нас словам) она завершает работу в состоянии q_0 (корректное завершение), и управляющая головка находится в конце (или перед началом) результирующего слова.

В рассмотренном примере можно сказать, что МТ с программой Π (или Π_1) *применима к слову 11* и результатом является именно слово 1111. В то же время необходимо обсудить ситуации НЕПРИМЕНИМОСТИ МТ к каким-либо словам.

Пример 4. Программа

$$\Pi : \begin{array}{l} q_1 1 q_1 1 R \\ q_1 0 q_1 1 R \end{array}$$

НЕ применима ни к какому слову на ленте, т.к. машина, начав работу, никогда не остановится и будет работать «вечно», желая «перепахать» всю «целину» из нулей и заменить эти нули единичками.

Пример 5. Программа

$$\Pi : q_1 1 q_1 1 R$$

также не применима ни к одному слову, т.к. наткнувшись на ноль, МТ не будет знать: что ей делать дальше ? Упрощенно говоря, МТ «зависнет», и такую ситуацию мы тоже относим к НЕПРИМЕНИМОСТИ МТ (ни к каким словам, и, в частности, к конкретным 11, 101 и т.п.).

Частный вывод: в программе, которую мы хотим считать применимой хотя бы к каким-либо словам, необходимо наличие команды с символом q_0 в третьей позиции. Необходимо, но, разумеется, НЕ достаточно !

Еще несколько примеров.

Пример-упражнение 6. Построить МТ, реализующую **удвоение** массива из единиц 1^n , т.е. составить программу, применимую ко всем словам 1^n и перерабатывающую их по схеме:

$$1^n \rightarrow 1^{2n}.$$

Пример-упражнение 7. Копирование $1^n \rightarrow 1^n 0 1^n$

Пример 8. $1^n 0^k 1^m \rightarrow 1^m 0^n 1^k$.

Предполагается выполнение такого (аналогичного) задания каждым студентом потока. Его выполнение войдет значимой частью в вашу итоговую оценку за семестр. Попутная подсказка: задание НЕ является тривиальным, программы могут содержать, ориентировочно, от 15 до 100 команд. Но время до конца семестра пока есть.

По поводу удвоения хочу начать обсуждения и надеюсь увидеть предложения с мест: как это реализуется ? Итак, имеем на ленте конфигурацию

$$q_1 11 \dots 1100 \dots,$$

и, наверное, как в разобранным примере, стоит написать первую команду программы в виде

$$q_1 1 q_1 1 R.$$

А дальше ?

Операции с машинами Тьюринга

Определение 3. Две МТ T_1, T_2 называются *эквивалентными* ($T_1 \simeq T_2$), если для любого слова P на ленте:

либо 1) обе машины не применимы к P ,

либо 2) обе применимы и результаты их применений равны $T_1(P) = T_2(P)$.

Определение 4. Пусть T - машина Тьюринга с программой Π . Если все R -сдвиги в этой программе заменить на L -сдвиги и наоборот, то получаемые машину и программу T^* и M^* называют *двойственными* к T и Π .

Определение 5. Пусть две МТ T_1, T_2 работают на одном и том же внешнем алфавите и имеют состояния

$$\{q'_0, q'_1, \dots, q'_r\} \quad \text{и} \quad \{q''_0, q''_1, \dots, q''_r\}$$

соответственно. Если в программе для T_1 все вхождения q'_0 заменить на q''_1 и объединить обе программы Π_1, Π_2 в одну, то получаемые таким образом программа и МТ называются *композицией* исходных программ и МТ.

Смысл композиции очень простой: выполнив одну «микрозадачу» первой машиной, передаем управление второй машине и выполняем ее программу.

Пример. В задаче удвоения слова 1^n (после простейших попыток решить ее «влёт») обычно приходит понимание о необходимости разбить ее на более простые подзадачи, а тем самым использовать композицию машин.

Подзадачи:

1) пробег по массиву единиц слева направо: $q_1 1 q_1 1 R$;

2) пробег по массиву единиц справа налево: $1 \dots 1 q_5 \rightarrow q_{10} 1 \dots 1$. Как реализовать такую микрозадачу ?

Замечание. Внутри программ для МТ, которые мы обсуждаем, заложен односторонний приоритет движения слева направо. Справа налево двигаться тоже можно, но с чуть большими сложностями.

$$q_5 0 q_6 0 L : 1 \dots 1 q_5 0 0 \dots \rightarrow 1 \dots 1 q_6 1 0 0 \dots$$

Определение 6. *Разветвление* машины T_0 на две машины T_1 и T_2 — это замена одной части вхождений заключительного состояния q_0 в программе

P_0 исходной машины на начальные состояния q'_1 машины T_1 , а другой части — на начальные состояния q'_1 машины T_2 .

Пример. Сравнить два массива 1^n и 1^k на ленте. Если длины равны, вывести на ленту 111, если не равны - вывести 101.